RideMove

Primer comité de arquitectura actualizado

Proyecto

Solicitud y gestión de transporte en Fusagasugá

Equipo

Laura Camila Díaz Rey
Fabián Sneider Martínez Moreno

Programa

Ingeniería de Software I
Universidad de Cundinamarca
22 de septiembre de 2026

Contexto del proyecto

Movilidad urbana y semiurbana

Situación actual

Las solicitudes dependen de mensajes de WhatsApp y llamadas para confirmar el servicio.

Necesidad

Un canal que reúna la solicitud, el conductor asignado, el estado del viaje y su historial.

Alcance

Aplicación móvil para pasajeros, conductores y administración.

Problema y solución

Un flujo de servicio trazable

Solicitud

El pasajero registra origen y destino. La plataforma crea un servicio pendiente.

Atención

Un conductor disponible acepta. Solo el asignado puede iniciar y finalizar el viaje.

Cierre

El pasajero consulta su historial y califica el viaje finalizado una sola vez.

Usuarios del sistema

Acceso según el rol

Pasajero	Solicita transporte, consulta el estado y el mapa del conductor, y califica el servicio.
Conductor	Gestiona disponibilidad y vehículo. Acepta o rechaza solicitudes y comparte ubicación durante el viaje.
Administrador	Consulta servicios y usuarios. Suspende o reactiva acceso y corrige vehículos de conductores.

Arquitectura hexagonal

El núcleo define las reglas del negocio

Entrada

Flutter envía HTTP a la API REST de Spring Boot. El controlador llama a RideUseCases.

Núcleo

RideService y las entidades Java validan roles, estados, autoría y disponibilidad. No importan frameworks.

Salida

SupabaseAdapter implementa los puertos de perfiles, viajes, vehículos y ubicación. PostgreSQL conserva la integridad.

Modelo de datos y seguridad

Información persistente y acceso controlado

Identidad y perfiles

Supabase Auth administra credenciales. profiles conserva rol, teléfono, disponibilidad y acceso.

Servicio

trips guarda estados, fechas y calificación. vehicles y locations guardan vehículo actual y última ubicación. rejections registra rechazos.

Protección

RLS filtra registros. Las RPC bloquean asignaciones incompatibles. El backend verifica el token y el rol real.

Tecnologías seleccionadas

Configuración implementada

Aplicación móvil	Flutter y Dart. geolocator para GPS. flutter_map y OpenStreetMap para visualizar al conductor.
Backend	Java 21 y Spring Boot 3.5.16. API REST con núcleo hexagonal y puertos independientes.
Datos y pruebas	Supabase Auth y PostgreSQL. Maven, JUnit, Mockito, ArchUnit y flutter_test.

Metodología y validación

Incrementos revisables con Scrum

Plan de trabajo

Sprints de una a dos semanas: usuarios, servicios, seguimiento y administración. Revisión del incremento con el comité.

Pruebas ejecutadas

24 pruebas Java del núcleo, arquitectura, HTTP y persistencia. Pruebas Flutter de interfaz, flujo y seguimiento.

Integración de datos

Flujo comprobado en Supabase con aislamiento RLS. Los registros de prueba se revirtieron al finalizar.

Alcance de esta entrega

Condiciones para la siguiente revisión

Implementado	RF-01 a RF-09, API hexagonal, mapa del conductor, ubicación en primer plano, esquema y pruebas.
Fuera del alcance inicial	Pagos electrónicos, bancos, tarifas complejas y asignación mediante inteligencia artificial.
Validación pendiente	Alojar la API con HTTPS, probar GPS en teléfono físico y medir rendimiento y concurrencia con usuarios reales.